RESEARCH-ARTICLE

Reference architecture knowledge representation: an experience

ELISA YUMI NAKAGAWA, University of São Paulo, Sao Paulo, SP, Brazil

JOSÉ CARLOS MALDONADO, University of São Paulo, Sao Paulo, SP, Brazil

Published: 13 May 2008

ICSE '08: International Conference on Software Engineering
May 13, 2008
Leipzig, Germany

Conference Sponsors:
SIGSOFT

SHARK '08: Proceedings of the 3rd international workshop on Sharing and reusing architectural knowledge (May 2008)
https://doi.org/10.1145/1370062.1370077
ISBN: 9781605580388

Reference Architecture Knowledge Representation: An Experience*

Elisa Yumi Nakagawa
Dept. of Computer Systems
University of São Paulo
São Carlos, SP, Brazil
elisa@icmc.usp.br

José Carlos Maldonado
Dept. of Computer Systems
University of São Paulo
São Carlos, SP, Brazil
jcmaldon@icmc.usp.br

ABSTRACT

Software architectures have played a significant role in determining the success of software systems. In spite of impact of the architectures to the software development and, as a consequence, to the software quality, there is not yet a consensus about which mechanisms work better to describe these architectures. In addition, despite the relevance of reference architectures as an artifact that comprises knowledge of a given domain and supports development of systems for that domain, issues related to their representation have not also had enough attention. In this perspective, this work intends to contribute with an experience of representing reference architectures aiming at easily sharing and reusing knowledge in order to develop software systems. A case study on software testing is presented illustrating our experience.

Categories and Subject Descriptors

D.2.11 [**Software Engineering**]: Software Architecture—*languages*

General Terms

Design, Documentation

Keywords

reference architecture, architectural view, architectural description

1. INTRODUCTION

Over the last decades software architecture has received increasing attention as an important subfield of software engineering [14]. Software architectures play a major role in determining system quality — performance, maintainability, and reliability, for instance — since they form the backbone for any successful software-intensive system [16]. In this perspective, reference architectures have emerged as an element that consolidates the knowledge of a specific domain. A reference architecture plays a dual role with regard to specific target software architectures [5]: (i) it generalizes and extracts common functions and configurations; and (ii) it provides a base for instantiating target systems that use that common base more reliably and cost effectively. Reference architectures for different domains — embedded systems, e-commerce, among others — can be found. In the same direction of use of reference architectures, the sharing and reuse of architectural knowledge have arisen as an promising research direction [1].

In order to correctly design and clearly document software architectures, ADLs (Architecture Description Languages)[1], have been proposed. As well-known examples of ADLs, ACME and Wright can be cited. However, documentation based on UML (Unified Modelling Language)[2] and architectural views have been investigated as a promising approach to document architectures [3]. According to IEEE 1471 - *Recommended practice for architectural description of software-intensive systems* (now also international standard ISO/IEC 42010) [7], an architectural description aggregates architectural views that represent or describe an entire system from a single perspective. The need for multiple views in architectural descriptions is widely recognized in the literature. However, authors differ on what views are needed and on appropriate methods/techniques for expressing each view [7]. In order to represent these views, recently, UML has been widely accepted in both industry and academia as a language for architectural description [8, 10]. In particular, UML 2.0 has created great expectations about the potential of the language to capture software architectures, since it makes possible to model large software systems and includes new concepts for specifying dynamically and statically interconnected patterns of object structures, as well as concepts for specifying complex object interaction patterns. These capabilities have been distilled from existing ADLs such as ACME.

In spite of diversity of works considering software architecture representation, specifically for reference architectures, there is also a lack of works that establish which views, techniques and languages work better to describe these architectures. Thus, in this paper, we present our experience in describing reference architectures in direction of sharing and reusing knowledge contained in these architectures.

*This work is supported by Brazilian funding agencies — FAPESP, Capes and CNPq — and QualiPSo European research project.

SHARK'08, May 13, 2008, Leipzig, Germany.

[1] http://www.sei.cmu.edu/architecture/adl.html
[2] http://www.uml.org

The remainder of this paper is organized as follows. In Section 2, we present our proposal to represent reference architectures. In Section 3 we present our experience through a case study on the software testing domain. In Section 4 we outline our conclusions and future directions.

2. REPRESENTING REFERENCE ARCHITECTURE KNOWLEDGE

The effective reuse of knowledge of reference architectures depends not only on raising the domain knowledge, but also documenting and communicating efficiently this knowledge through an adequate architectural description. Commonly, architectural views have been used to document architectural instances; however, we have used architectural views in order to represent reference architectures. Reference architectures have higher abstraction level than architectural instances; therefore, low level representation, for instance, code view of Siemens' Four View or implementation and data views, are not adequate to be used. Thus, UML techniques and three views have been selected in our work to represent reference architectures:

- module view shows the structure of the software in terms of code units and packages and classes can be used to represent this view, as well as containment, specialization/generalization, and dependency relations. For this view, UML class diagram is an adequate technique;
- runtime view shows the structure of the software system when it is executing. The elements that compose this view are components that have runtime presence, datas and connectors. UML component diagram can be used to represent this view; and
- deployment view describes the machines, software that is installed on that machines and network connections that are used by the software systems. Thus, an adequate technique is UML deployment diagram.

Additionally, we have observed that only these views and UML diagrams are not sufficient to completely represent reference architectures, since these diagrams do not provide elements to explain each concept/term that is present in these architectures. We have proposed additional view, named conceptual view, that aims at describing each concept of the domain. For describing this view, ontologies, controlled vocabularies, taxonomies, thesauri, concept maps, among others can be used as a supporting element to describe the terminology related to that domain. Besides that, although recently AOP (Aspect-Oriented Programming) [9] has been considered in the early phases of software life cycle, the representation of aspects in architectural level is still an open research line, as recently discussed in [4]. In order to represent adequately aspect-oriented reference architecture (i.e. architectures that contain aspects in their structures), as it is in our case, extensions to UML techniques have also been proposed [12].

3. CASE STUDY

In order to illustrate our idea, we present the description of a reference architecture for software testing domain, called RefTEST (Reference Architecture for Software Testing Tools) [13]. RefTEST was proposed since testing automation is an important issue related to software product quality. However, testing tools have almost always been implemented individually and independently, and issues related to reuse of knowledge about how to develop these tools have been ignored. RefTEST is part of a broader scope that involves the establishment of reference architectures for the software engineering domain [11]. RefTEST is a specialization of RefASSET (Reference Architecture for Software Engineering Tools), a more general architecture that supports the development of SEEs (Software Engineering Environments). In short, RefASSET is based on international standard ISO/IEC 12207 (Information Technology - Software Life Cycle Processes) [6] and architectures of interactive systems (architectural pattern MVC (Model-View-Controller) and 3-tiers architecture[3]). Besides that, this architecture is strongly based on SoC (each concern corresponds to a software engineering activity), aiming at providing reusability, maintainability, and capability of evolution to the environments built based on this architecture. Furthermore, we have explored the use of aspects (from AOP) in the early life cycle phases, i.e. in architectural design phase, resulting in an aspect-based reference architecture. As a consequence, RefTEST inherits all RefASSET's characteristics. It inherits also the relationship among primary modules and modules that automate organizational and supporting activities, such as documentation and configuration management. These modules (that implement organizational and supporting activities) present crosscutting characteristic, since they automate activities spread throughout or tangled with others software engineering activities [11], in the same perspective considered by the AOSD (Aspect-Oriented Software Development) community for activities that have this characteristic. For instance, documentation, a supporting activity, is performed from requirement specification to maintenance and, therefore, it is a crosscutting activity.

In order to adequately document and communicate the reference architecture, we have used module, runtime and deployment views and UML 2.0. The module view, presented in Figure 1, shows the package `testing_tool` that contains the core of testing tools. Other packages (`supporting_crosscutting_modules`, `organizational_crosscutting_modules` and `general_crosscutting_modules`) contain modules that automate crosscutting activities. The modules contained in these packages use aspects in their structures and use these aspects to communicate with other modules. Thus, there are dependency relations, labelled with <<`crosscut`>>, among packages that contain modules that implement crosscutting activities and other packages. For example, documentation module is composed by aspects that crosscut other modules, providing functionalities related to documentation.

The runtime view, illustrated in Figure 2, shows the interaction among base module (`Testing`), persistence module (`Persistence`), security module (`Security`) and crosscutting modules (for instance, `Documentation`, `Planning_Management` and `Configuration_Management`). It is important to highlight that since the modules in this architecture communicate using aspects, a special notation for required and provided interfaces is a need. In our case, we have proposed a filled cycle to represent the interface of these modules. We have

[3]http://www.sei.cmu.edu/str/descriptions/threetier.html

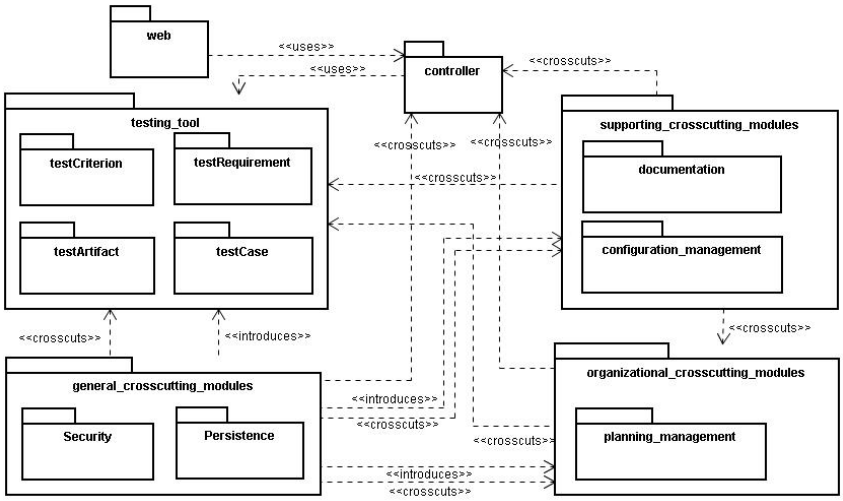

Figure 1: Module View of RefTEST

also proposed a half-square, attached by a solid line to the module that is crosscuted or affected by aspects [12].

The deployment view, illustrated in Figure 3, presents a possible way to deploy testing tools developed using RefTEST. Through client machine (`Internet User` and `Intranet Admin`), the tester uses a web browser to have access to testing tool. The web server (`Web server`) provides answers to the requests of the client machines. The application server (`Application server`) contains all functionalities of the testing tool. The database server (`Database server`) manages datas that are produced and consumed by testing tools. Other configurations for this view are also possible, for instance, considering only a unique machine running the web server and application server.

Through our experience, we have observed that the use of UML diagrams made easier reading and understanding the reference architectures. Besides that, in order to represent conceptual view, we have explored the use of an ontology. An ontology basically consists of concepts and relations, as well as their definitions, properties and constraints expressed by means of axioms [15]. Thus, an ontology for software testing domain — OntoTest [2] — has been used to describe better RefTEST. For example, through this ontology, the concept `testCriterion` into `testing_tool` package (in Figure 1) refers to "a systematic way to obtain a test case set that is effective to identify software errors". In practice, this concept corresponds to a submodule that aggregates the steps of a testing criterion in order to obtain a test case set.

In order to support the software development based on reference architecture, a process, named ProSA (Process based on Software Architecture) [11], was established. Thus, based on RefTEST, we have developed JaBUTi/Web (Java Bytecode Understanding and TestIng for Web), a testing tool that supports structural testing — All-node, All-edge, All-uses and All-potential-uses — of Java applications. This tool has been implemented in Java[4] and deployed in web platform. We have also developed testing support modules — a documentation module and a configuration management module — based on this architecture.

[4]http://java.sun.com/

4. CONCLUSIONS

Readability and understandability of reference architectures are fundamental to their dissemination and, as a consequence, to posterior validation and evolution. We have presented an experience in the use of architectural views and UML techniques in order to describe reference architectures. In essence, we have observed that not all views are adequate to represent reference architectures, considering their higher abstraction level. Additionally, only the use of known architectural views is not sufficient to completely describe a reference architecture, requiring some extra artifact — describing the conceptual view — such as an ontology that has been used in our experience. The use of these views to describe reference architectures seems to have been an appropriate choice. However, more studies in order to investigate which and how others views, if any, can contribute to represent reference architectures need to be conducted. As future perspective, we intend to apply results of this experience in a reference architecture for software engineering domain (the RefASSET).

5. REFERENCES

[1] P. Avgeriou, P. Kruchten, P. Lago, P. Grisham, and D. Perry. Sharing and reusing architectural knowledge – architecture, rationale, and design intent. *29th Int. Conf. on Software Engineering (ICSE'07 Companion)*, 00:109–110, 2007.

[2] E. F. Barbosa, E. Y. Nakagawa, and J. C. Maldonado. Towards the establishment of an ontology of software testing. In *Proc. of the 18th Int. Conf. on Soft. Engineering and Knowledge Engineering (SEKE'06)*, San Francisco Bay/USA, July 2006.

[3] P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Iveers, R. Little, R. Nord, and J. Stafford. *Documenting Software Architecture: Views and Beyond*. Addison-Wesley, Boston, MA, 3 edition, 2003.

[4] P. Clements, D. Emery, R. Hilliard, and P. Kruchten. Aspects in architectural description: report on a 1st workshop at AOSD 2007. *SIGSOFT Softw. Eng. Notes*, 32(4):33–35, 2007.

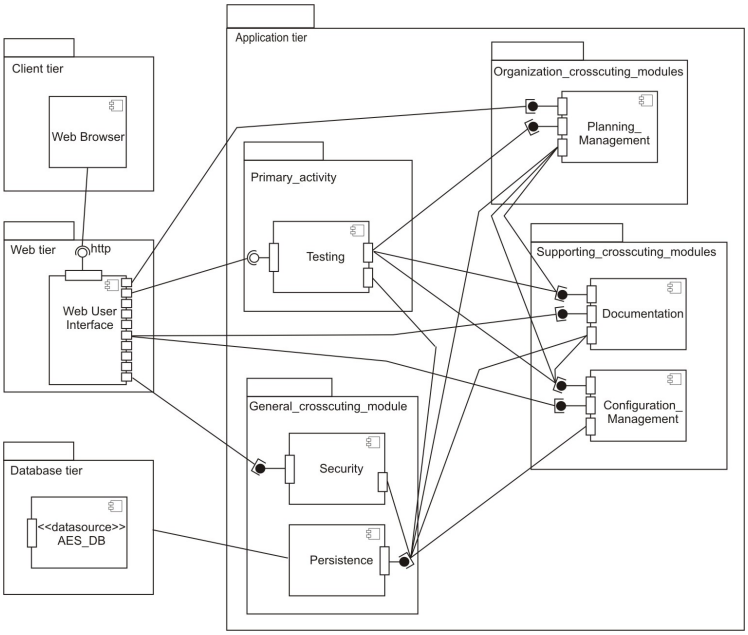

Figure 2: Runtime View of RefTEST

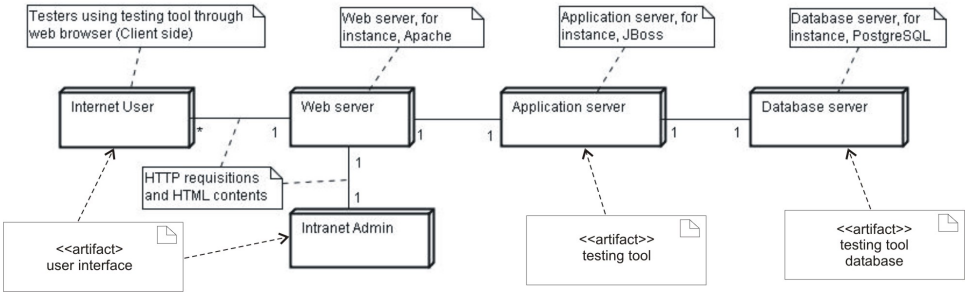

Figure 3: Deployment View of RefTEST

[5] B. P. Gallagher. Using the architecture tradeoff analysis method to evaluate a reference architecture: A case study. Technical Report CMU/SEI-2000-TN-007, 2000.

[6] ISO. ISO/IEC 12207. Information technology – software life-cycle processes, 1995.

[7] ISO. ISO/IEC 42010 - Recommended practice for architectural description of software-intensive systems, 2007.

[8] J. Ivers, P. Clements, D. Garlan, R. Nord, B. Schmerl, and J. R. O. Silva. Documenting component and connector views with UML 2.0. Technical report, 2004. CMU/SEI-2004-TR-008.

[9] G. Kiczales, J. Irwin, J. Lamping, J. Loingtier, C. Lopes, C. Maeda, and A. Menhdhekar. Aspect-oriented programming. In *Proc. of the Eur. Conf. on Object-Oriented Programming*, pages 220–242, Berlin, Heidelberg, and New York, 1997.

[10] N. Medvidovic, D. S. Rosenblum, D. F. Redmiles, and J. E. Robbins. Modeling software architectures in the Unified Modeling Language. *ACM Trans. Softw. Eng. Methodol.*, 11(1):2–57, 2002.

[11] E. Y. Nakagawa. *A Contribution to the Architectural Design of Software Engineering Environments*. PhD thesis, University of São Paulo, São Carlos, SP, Brazil, Aug. 2006. (in Portuguese).

[12] E. Y. Nakagawa and J. C. Maldonado. Representing aspect-based architecture of software engineering environments. In *1st Workshop on Aspects in Architectural Description, 6th Int. Conf. on AOSD*, Vancouver, Canada, Mar. 2007.

[13] E. Y. Nakagawa, A. S. Simão, F. Ferrari, and J. C. Maldonado. Towards a reference architecture for software testing tools. In *Proc. of the 19th Int. Conf. on Software Engineering and Knowledge Engineering (SEKE'2007)*, Boston, USA, July 2007.

[14] M. Shaw and P. Clements. The golden age of software architecture. *IEEE Software*, 23(2):31–39, Mar/Apr 2006.

[15] M. Uschold and M. Grüninger. Ontologies: principles, methods, and applications. *Knowledge Engineering Review*, 11(2):93–155, 1996.

[16] A. I. Wasserman. Towards a discipline of software engineering. *IEEE Software*, 13(6):23–31, Nov. 1996.